

Infosys Java + Spring Boot Mock Interview

100 Questions • Detailed Model Answers • Follow-up Cross-Questions • Coding & SQL

Target: Java/Spring Boot developer with around 3 years of experience. Use this as a two-day rehearsal: answer aloud before checking the model answer.

PART A — 100 QUESTIONS ONLY

1. Tell me about yourself and your current project.
2. What are the four pillars of OOP?
3. Explain abstraction vs encapsulation.
4. Overloading vs overriding?
5. Is Java pass-by-value or pass-by-reference?
6. == vs equals()?
7. Why is String immutable?
8. String vs StringBuilder vs StringBuffer?
9. Abstract class vs interface?
10. What are default methods in interfaces?
11. final, finally, and finalize()?
12. Checked vs unchecked exceptions?
13. Can we have try without catch?
14. What happens if an exception is thrown in finally?
15. What is an immutable class?
16. Shallow copy vs deep copy?
17. Static vs instance members?
18. Stack vs heap?
19. What is garbage collection?
20. throw vs throws?
21. Explain Java Collections Framework.
22. Why isn't Map a child of Collection?
23. ArrayList vs LinkedList?
24. HashSet vs TreeSet vs LinkedHashSet?
25. HashMap vs Hashtable?
26. Explain HashMap internal working.
27. What is a HashMap collision?
28. Why must equals/hashCode be consistent?
29. HashMap vs ConcurrentHashMap?
30. Comparable vs Comparator?
31. What are Java 8 features?
32. What is a functional interface?
33. Predicate vs Function vs Consumer vs Supplier?
34. What is a lambda expression?
35. What is Stream API?
36. map() vs filter()?
37. map() vs flatMap()?
38. Intermediate vs terminal operations?
39. What is lazy evaluation?
40. What is Optional?

41. `orElse()` vs `orElseGet()`?
42. What does `reduce()` do?
43. How does `groupBy()` work?
44. `findFirst()` vs `findAny()`?
45. `stream()` vs `parallelStream()`?
46. What is Spring Framework?
47. Spring vs Spring Boot?
48. What is IoC?
49. What is Dependency Injection?
50. Constructor injection vs field injection?
51. What is a Spring Bean?
52. Explain Bean lifecycle.
53. `@Component` vs `@Service` vs `@Repository`?
54. `@Bean` vs `@Component`?
55. What is `@Autowired`?
56. What if two beans implement the same interface?
57. What does `@SpringBootApplication` contain?
58. What is auto-configuration?
59. What are Spring Boot starters?
60. Explain Spring Boot startup.
61. `application.properties` / `application.yml`?
62. `@Value` vs `@ConfigurationProperties`?
63. What are Spring Profiles?
64. What is Actuator?
65. How do you debug startup failure?
66. `@Controller` vs `@RestController`?
67. Explain complete REST request flow.
68. `@PathVariable` vs `@RequestParam` vs `@RequestBody`?
69. What is `ResponseEntity`?
70. What is `DispatcherServlet`?
71. How do you implement global exception handling?
72. How do you validate request data?
73. Common HTTP status codes?
74. What is REST?
75. JPA vs Hibernate?
76. What is an Entity?
77. Explain JPA relationships.
78. Lazy vs eager loading?
79. What is N+1?
80. What is cascade?
81. `orphanRemoval` vs `cascade REMOVE`?

82. What is @Transactional?
83. REQUIRED vs REQUIRES_NEW?
84. What is transaction isolation?
85. What happens when a transaction fails?
86. Authentication vs authorization?
87. Explain JWT.
88. Explain JWT flow in Spring Security.
89. What is SecurityFilterChain?
90. 401 vs 403?
91. Monolith vs microservices?
92. What is an API Gateway?
93. What is service discovery?
94. What is a circuit breaker?
95. How do microservices communicate?
96. Process vs thread?
97. Runnable vs Callable?
98. synchronized vs volatile?
99. What is ExecutorService?
100. How do you handle simultaneous DB updates?

PART B — DETAILED MODEL ANSWERS + FOLLOW-UPS

1. Tell me about yourself and your current project.

Model answer: I'm a Java Full Stack Developer with around three years of experience, primarily working with Java, Spring Boot, REST APIs, Angular and relational databases. In my current project I work across backend and frontend: REST APIs, business logic, validation, JPA/Hibernate database operations and Angular UI/API integration. I also work on debugging, testing and issue resolution. I'm now looking for an opportunity where I can deepen my Java/Spring Boot expertise and work on larger enterprise applications.

Cross-questions: What exactly did YOU develop? Explain one API end-to-end. Which database? How did you handle validation, exceptions and testing?

2. Four pillars of OOP.

Model answer: Encapsulation bundles state and behavior and controls access. Abstraction exposes what is needed while hiding implementation. Inheritance allows a class to derive from another. Polymorphism lets the same reference/interface invoke different implementations at runtime.

Cross: Give a real project example of each.

3. Abstraction vs encapsulation.

Model answer: Abstraction is about what an object exposes; encapsulation is about protecting and controlling its internal state. An interface such as `PaymentService` exposes a contract, while private fields with controlled methods demonstrate encapsulation.

Cross: Can an abstract class provide both? Yes.

4. Overloading vs overriding.

Model answer: Overloading uses the same method name with different parameter lists and is resolved at compile time. Overriding replaces inherited behavior and is resolved at runtime. Return type alone cannot overload a method.

Cross: Can static methods be overridden? They are hidden, not dynamically overridden.

5. Java pass-by-value.

Model answer: Java is always pass-by-value. For an object, the copied value is the reference value. You can mutate the referenced object, but reassigning the parameter does not change the caller's reference.

Cross: Demonstrate with an object parameter.

6. == vs equals().

Model answer: For primitives, `==` compares values. For objects, `==` compares references. `equals()` compares logical equality when the class implements it appropriately. `String` is the classic example.

Cross: Why does this matter for `HashMap` keys?

7. String immutability.

Model answer: `String` cannot be changed after creation. This supports safe sharing/string pooling, thread-safety and stable hash behavior, which is useful when `Strings` are map keys. Concatenation creates another `String`.

Cross: Why is a mutable `HashMap` key dangerous?

8. String vs StringBuilder vs StringBuffer.

Model answer: `String` is immutable. `StringBuilder` is mutable and usually preferred for repeated string manipulation in normal single-threaded code. `StringBuffer` is synchronized and generally has more overhead.

Cross: What would you use in a loop building a large string? `StringBuilder`.

9. Abstract class vs interface.

Model answer: An abstract class can have state, constructors, concrete methods and abstract methods. An interface defines a contract and supports multiple implementation inheritance; modern interfaces can also contain default, static and private methods.

Cross: Can an interface have implementation? Yes.

10. Default methods.

Model answer: A default method gives an interface a method implementation. It helped evolve interfaces without breaking every existing implementation. Conflicting defaults must be resolved by the implementing class.

Cross: How do you resolve the conflict? Override the method.

11. final/finally/finalize.

Model answer: final can prevent reassignment, overriding or inheritance depending on usage. finally is an exception-cleanup block. finalize was a legacy GC mechanism and should not be relied upon for cleanup in modern Java.

Cross: What if try and finally both throw? The finally exception can obscure the original exception.

12. Checked vs unchecked exceptions.

Model answer: Checked exceptions must be handled or declared. Examples include IOException. Unchecked exceptions extend RuntimeException, such as NullPointerException and IllegalArgumentException.

Cross: Should every exception be caught? No—handle where meaningful.

13. try without catch.

Model answer: Yes. try can be followed by finally without catch, typically for cleanup that must occur regardless of success or failure.

Cross: Why use try-with-resources? It manages Closeable resources safely.

14. Exception in finally.

Model answer: If try throws and finally also throws, the finally exception can hide the original exception. Avoid unrelated exceptions from cleanup code.

Cross: Prefer try-with-resources for resource cleanup.

15. Immutable class.

Model answer: Make the class final, fields private and final, provide no setters, defensively copy mutable inputs, and never expose mutable internal state. List.copyOf is useful for immutable list state.

Cross: Why defensive copying? Otherwise callers can mutate internal state.

16. Shallow vs deep copy.

Model answer: Shallow copy duplicates the outer object but can share nested references. Deep copy duplicates nested mutable objects too, so changes are independent.

Cross: Give an Employee/Address example.

17. Static vs instance.

Model answer: Static members belong to the class; instance members belong to an object. A static method has no implicit object reference and cannot directly access instance state.

Cross: Can static access instance data? Only through an object reference.

18. Stack vs heap.

Model answer: Each thread has its own stack containing method frames and local data. Objects are generally allocated on the shared heap and managed by garbage collection.

Cross: Why can shared heap state cause race conditions?

19. Garbage collection.

Model answer: Java automatically reclaims memory for objects that are no longer reachable. System.gc() is only a request/hint, not a guaranteed immediate collection.

Cross: Can application code force GC reliably? No.

20. throw vs throws.

Model answer: throw actually throws an exception. throws declares that a method may propagate exceptions.

Cross: Can throws list multiple exceptions? Yes.

21. Collections Framework.

Model answer: Major abstractions include List, Set, Queue and Map. Examples include ArrayList, LinkedList, HashSet, TreeSet, PriorityQueue, HashMap and ConcurrentHashMap. Map is separate because it models key-value mappings.

Cross: Best default list for random access? ArrayList.

22. Why Map isn't Collection.

Model answer: Collection represents individual elements. Map represents key-value associations, so its API and abstraction are different.

Cross: Does Map extend Collection? No.

23. ArrayList vs LinkedList.

Model answer: ArrayList is array-backed with efficient indexed access and may require shifting on middle insertion/removal. LinkedList is node-based and indexed access is $O(n)$; insertion/removal is efficient when the node/position is already known.

Cross: What is your normal default? Usually ArrayList.

24. Set variants.

Model answer: HashSet prevents duplicates without guaranteed order. LinkedHashSet preserves insertion order. TreeSet keeps elements sorted using natural ordering or a Comparator.

Cross: How does TreeSet determine duplicates? By comparison returning zero.

25. HashMap vs Hashtable.

Model answer: HashMap is not synchronized and permits null key/value. Hashtable is legacy, synchronized and does not permit nulls. Modern concurrent code generally uses ConcurrentHashMap.

Cross: Why not use Hashtable in new code? Legacy/coarse-grained design.

26. HashMap internal working.

Model answer: HashMap hashes the key, calculates a bucket and stores an entry there. `get()` computes the hash, finds the bucket and uses `hash plus equals()` to identify the key. Collisions are handled within buckets; modern Java can treeify heavily-collided buckets under suitable conditions.

Cross: Explain resizing, load factor, collision and `equals/hashCode`.

27. HashMap collision.

Model answer: A collision occurs when different keys map to the same bucket. HashMap keeps multiple entries in that bucket and uses equality to find the correct key.

Cross: Can different keys have the same `hashCode`? Yes.

28. equals/hashCode consistency.

Model answer: If `a.equals(b)` is true, their hash codes must be equal. Equal hash codes do not imply equality. HashMap uses the hash to locate a bucket and `equals` to confirm the key.

Cross: What if a key's hash-related fields are mutated after insertion? Lookup can fail.

29. HashMap vs ConcurrentHashMap.

Model answer: HashMap is not thread-safe. ConcurrentHashMap supports concurrent access and atomic map operations such as `putIfAbsent`, `compute` and `merge`. It does not permit null keys or values. Multi-step business operations still need appropriate atomic design.

Cross: Does ConcurrentHashMap make every sequence of operations atomic? No.

30. Comparable vs Comparator.

Model answer: Comparable defines natural ordering inside the class. Comparator defines external/custom orderings and allows multiple strategies.

Cross: Why is Comparator useful for multiple business sort orders?

31. Java 8 features.

Model answer: Know lambdas, functional interfaces, Streams, Optional, method references, default/static interface methods and java.time.

Cross: Give one real project use of Streams.

32. Functional interface.

Model answer: An interface with exactly one abstract method. It may still have default and static methods. @FunctionalInterface lets the compiler enforce this.

Cross: Examples? Predicate, Function, Consumer, Supplier, Runnable, Callable.

33. Predicate/Function/Consumer/Supplier.

Model answer: Predicate takes input and returns boolean. Function maps input to output. Consumer takes input and returns nothing. Supplier takes no input and supplies a value.

Cross: Give a Stream example of each.

34. Lambda.

Model answer: A lambda is a compact implementation of a functional interface, such as $(a,b) \rightarrow a+b$. It reduces anonymous-class boilerplate.

Cross: Does a lambda need a target functional-interface type? Yes.

35. Stream API.

Model answer: A Stream is a pipeline for processing data, not a collection. A typical pipeline is filter → map → sorted → terminal operation.

Cross: Can a stream be reused after a terminal operation? No.

36. map vs filter.

Model answer: filter selects elements matching a predicate. map transforms each element into another value.

Cross: Which changes values? map.

37. map vs flatMap.

Model answer: map can produce nested structures such as `Stream<List<String>>`. flatMap transforms and flattens nested streams into one stream.

Cross: Example: flatten employees' lists of skills.

38. Intermediate vs terminal.

Model answer: Intermediate operations include filter, map, flatMap and sorted. Terminal operations include toList, collect, count, reduce and findFirst. A terminal operation triggers processing.

Cross: Are intermediate operations generally lazy? Yes.

39. Lazy evaluation.

Model answer: Intermediate Stream operations normally execute only when a terminal operation is invoked. This lets the pipeline avoid unnecessary work, especially with operations such as limit/findFirst.

Cross: Why can findFirst stop processing early?

40. Optional.

Model answer: Optional represents a value that may be absent. It makes absence explicit and supports operations such as map, orElse, orElseGet and orElseThrow. Avoid blindly using get().

Cross: Should every entity field be Optional? No.

41. orElse vs orElseGet.

Model answer: orElse evaluates its argument eagerly. orElseGet invokes the supplier only when the Optional is empty. For expensive defaults, orElseGet avoids unnecessary work.

Cross: What if the default is a cheap constant? Difference may be negligible.

42. reduce.

Model answer: reduce combines stream elements into one result, e.g. `numbers.stream().reduce(0, Integer::sum)`. For parallel processing, an appropriate associative operation matters.

Cross: What is the identity value? The starting value that does not change the result.

43. groupingBy.

Model answer: `groupingBy` creates a Map based on a classifier. It can combine downstream collectors such as counting, mapping, `maxBy` and `summingInt`.

Cross: How do you count employees by department? `groupingBy(Employee::getDepartment, counting())`.

44. findFirst vs findAny.

Model answer: `findFirst` respects encounter order for ordered streams. `findAny` can return any match and is more flexible for parallel execution.

Cross: Which expresses an ordering requirement? `findFirst`.

45. stream vs parallelStream.

Model answer: `stream` is sequential; `parallelStream` can use parallel execution. Parallelism is not automatically faster—consider task size, overhead, shared state, ordering and data source.

Cross: Why can small tasks become slower? Parallel overhead.

46. Spring Framework.

Model answer: Spring provides IoC/DI, AOP, transaction management, web capabilities and integrations with persistence, security and other technologies.

Cross: Which Spring modules do you use? Answer from your actual project.

47. Spring vs Spring Boot.

Model answer: Spring is the core framework. Spring Boot simplifies application setup with auto-configuration, starters, embedded servers and externalized configuration.

Cross: Does Boot replace Spring? No.

48. IoC.

Model answer: Inversion of Control means the Spring container manages object creation and dependencies rather than each class manually constructing its collaborators.

Cross: What is `ApplicationContext`? A central Spring IoC container.

49. Dependency Injection.

Model answer: A class declares its dependencies and the framework supplies them. Constructor injection makes dependencies explicit and testable.

Cross: Main benefits? Loose coupling and testability.

50. Constructor vs field injection.

Model answer: Constructor injection makes required dependencies explicit, supports final fields and simplifies unit testing. It is generally preferred over field injection.

Cross: How would you model optional dependencies? Explicitly, according to design.

51. Spring Bean.

Model answer: A Spring Bean is an object managed by the Spring IoC container. It can be discovered as a component or created with a `@Bean` method.

Cross: Are all Java objects Spring beans? No.

52. Bean lifecycle.

Model answer: Conceptually: instantiate → inject dependencies → initialization callbacks/`BeanPostProcessors` → ready for use → destruction callbacks at context shutdown. `@PostConstruct` and `@PreDestroy` are common lifecycle hooks.

Cross: Why use @PostConstruct? Initialize after dependencies are ready.

53. Component/Service/Repository.

Model answer: They are stereotype annotations. Component is generic; Service communicates business/service-layer intent; Repository communicates persistence and supports exception translation.

Cross: Does @Service itself start a transaction? No.

54. @Bean vs @Component.

Model answer: @Component is discovered through scanning. @Bean is an explicit factory method, useful for third-party classes or detailed construction/configuration.

Cross: Can @Bean method parameters be injected? Yes.

55. @Autowired.

Model answer: It tells Spring to resolve and inject a dependency. With one constructor, explicit @Autowired is generally unnecessary in modern Spring.

Cross: Multiple candidates? @Primary or @Qualifier.

56. Two implementations.

Model answer: Spring cannot choose automatically when multiple candidates are equally valid. Use @Primary, @Qualifier, or intentionally inject all implementations as a collection/map.

Cross: When would you inject List<Service>? Strategy/plugin style processing.

57. @SpringBootApplication.

Model answer: It combines @Configuration, @EnableAutoConfiguration and @ComponentScan.

Cross: What does component scanning find? Eligible Spring components in the scan path.

58. Auto-configuration.

Model answer: Boot conditionally configures common components based on classpath, properties, existing beans and conditions. Your explicit configuration can influence or replace auto-configured beans.

Cross: How can you debug conditional configuration? Debug/condition evaluation information.

59. Starters.

Model answer: Starters are dependency bundles such as spring-boot-starter-web, data-jpa, security and test. They simplify dependency management and version compatibility.

Cross: What are transitive dependencies? Dependencies brought in by another dependency.

60. Startup.

Model answer: SpringApplication.run builds the application context, loads configuration, scans components, applies auto-configuration, creates beans and starts the embedded server for a web application. With the common web starter, Tomcat is the default servlet container unless changed.

Cross: What happens before the app is ready? Context creation, bean initialization and server startup.

61. properties/yml.

Model answer: External configuration stores datasource, server, logging and application settings outside business code. Environment-specific values can be supplied externally.

Cross: Where should production secrets live? Secret management/environment configuration, not source control.

62. @Value vs ConfigurationProperties.

Model answer: @Value is convenient for individual properties. @ConfigurationProperties is better for a structured, type-safe group of related configuration values.

Cross: Which scales better for many settings? ConfigurationProperties.

63. Profiles.

Model answer: Profiles allow environment-specific configuration and bean activation, such as dev, test and prod. They help separate deployment configuration from application logic.

Cross: How do you activate a profile? Configuration/environment/deployment settings.

64. Actuator.

Model answer: Actuator exposes operational endpoints such as health and metrics depending on configuration. It supports monitoring and diagnostics.

Cross: Should all endpoints be public? No; secure/expose only what is needed.

65. Startup debugging.

Model answer: Start with the first meaningful root exception in logs. Check bean creation, dependency compatibility, profiles/configuration, database connectivity, port binding and recent changes. Reproduce and fix the root cause.

Cross: How would you investigate a circular dependency? Refactor first; don't hide bad design with lazy workarounds.

66. Controller vs RestController.

Model answer: @Controller is commonly used for MVC views. @RestController combines controller behavior with response-body semantics for REST responses.

Cross: Can @Controller return JSON? Yes, with @ResponseBody.

67. REST request flow.

Model answer: Angular HttpClient sends HTTP → web server/DispatcherServlet → handler mapping → Controller → Service → Repository → JPA/Hibernate → database. The result travels back, Jackson serializes it to JSON and Angular receives the response.

Cross: Where should business logic live? Service layer.

68. PathVariable/RequestParam/RequestBody.

Model answer: PathVariable identifies a resource, e.g. /employees/10. RequestParam handles query parameters, e.g. ?page=0. RequestBody carries JSON/XML request payload.

Cross: Pagination? Usually query parameters.

69. ResponseEntity.

Model answer: ResponseEntity gives explicit control over status, headers and body. It is useful for returning 201, 204, custom headers, etc.

Cross: When would you return 204? Successful operation with no response body.

70. DispatcherServlet.

Model answer: It is Spring MVC's front controller. It receives HTTP requests and delegates to the appropriate handler/controller through MVC infrastructure.

Cross: What helps select the controller method? Handler mappings.

71. Global exception handling.

Model answer: Use @RestControllerAdvice and @ExceptionHandler to centralize exception-to-HTTP-response mapping and return consistent error payloads.

Cross: What belongs in an error response? Useful client-facing information without sensitive internals.

72. Validation.

Model answer: Use Jakarta Bean Validation annotations such as @NotBlank, @Email and @Positive on DTOs, then @Valid in the controller. Handle validation exceptions centrally.

Cross: Why DTO validation? Keeps API contracts separate from persistence models.

73. HTTP statuses.

Model answer: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 authentication problem, 403 authenticated but forbidden, 404 Not Found, 409 Conflict, 500 server error.

Cross: Give a 409 example: unique constraint/business-state conflict.

74. REST.

Model answer: REST is an architectural style emphasizing resources, stateless interactions, uniform interface and cacheability where appropriate. HTTP is a protocol; REST is not synonymous with HTTP.

Cross: Is every HTTP API RESTful? No.

75. JPA vs Hibernate.

Model answer: JPA is a specification for ORM/persistence. Hibernate is a popular implementation/provider of JPA.

Cross: What does Hibernate ultimately use to communicate with a relational DB? JDBC/driver.

76. Entity.

Model answer: An entity is a persistent object mapped to a database table, normally annotated with `@Entity` and an `@Id`.

Cross: Why is an identifier needed? JPA needs entity identity.

77. Relationships.

Model answer: `ManyToOne` is common for many records pointing to one parent; `OneToMany` represents a collection; `ManyToMany` uses a join table. In bidirectional mappings, `mappedBy` marks the inverse side and the owning side controls the relationship mapping.

Cross: In a typical `Employee`→`Department` relationship, which side commonly owns the FK? `Employee/ManyToOne`.

78. Lazy vs eager.

Model answer: Lazy defers association loading. Eager loads according to the fetch strategy. Don't make everything eager; it can create huge object graphs and unnecessary DB work.

Cross: What can happen when a lazy association is accessed outside the persistence context? `LazyInitializationException`.

79. N+1.

Model answer: One query loads N parent rows and then N additional association queries are triggered. Use fetch joins, `EntityGraph`, batch fetching or projections as appropriate.

Cross: Is EAGER a universal fix? No.

80. Cascade.

Model answer: JPA cascade propagates operations such as `PERSIST`, `MERGE` and `REMOVE` from one entity to related entities. Use it deliberately, especially for deletes.

Cross: Is JPA cascade identical to DB `ON DELETE CASCADE`? No.

81. orphanRemoval vs cascade REMOVE.

Model answer: Cascade `REMOVE` propagates parent deletion. `orphanRemoval` can remove a child when it is removed from the parent's relationship, fitting aggregate-style ownership.

Cross: When useful? Child has no meaningful independent lifecycle.

82. @Transactional.

Model answer: It defines a transaction boundary. Spring commonly applies transaction behavior through proxies/interceptors. Default rollback is generally for unchecked exceptions and `Errors`; checked exceptions need explicit configuration when rollback is required.

Cross: Why can self-invocation bypass it? `Direct this.method()` bypasses the Spring proxy.

83. REQUIRED vs REQUIRES_NEW.

Model answer: `REQUIRED` joins an existing transaction or creates one. `REQUIRES_NEW` suspends the existing transaction and starts an independent one. `REQUIRES_NEW` can increase connection-pool usage.

Cross: Does outer rollback undo `REQUIRES_NEW` work? Not merely because the outer transaction rolled back.

84. Isolation.

Model answer: Common levels are READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ and SERIALIZABLE. Know dirty reads, non-repeatable reads and phantom reads.

Cross: Which prevents dirty reads? READ_COMMITTED and stronger.

85. Transaction failure.

Model answer: If a configured rollback condition occurs, the transaction is rolled back. Spring commonly rolls back on RuntimeException/Error by default. Use rollbackFor when checked exceptions should cause rollback.

Cross: How do you configure rollbackFor? @Transactional(rollbackFor=...).

86. Authentication vs authorization.

Model answer: Authentication answers who the user is. Authorization answers what the authenticated user is allowed to do.

Cross: Example: authenticated USER can GET; ADMIN can DELETE.

87. JWT.

Model answer: JWT commonly contains Header.Payload.Signature. The payload is Base64URL-encoded and readable; the signature protects integrity/authenticity. A normal signed JWT is not encrypted, so don't store secrets in it.

Cross: Common claims? sub, exp, iat, iss, aud, roles/authorities.

88. JWT flow.

Model answer: Login verifies credentials. On success a JWT is issued. Subsequent requests send Authorization: Bearer token. Security filters validate the token, establish authentication and authorization rules decide access before controller execution.

Cross: Where should token validation happen? In the security filter chain.

89. SecurityFilterChain.

Model answer: It defines Spring Security's request-processing filters and authorization configuration. Authentication filters can establish the SecurityContext before protected endpoints execute.

Cross: Where would a custom JWT filter be inserted? At an appropriate position in the chain.

90. 401 vs 403.

Model answer: 401 means authentication is missing/invalid. 403 means the request is authenticated but the principal is not authorized to access the resource.

Cross: Give one example of each.

91. Monolith vs microservices.

Model answer: Monolith is one deployable application and is simpler initially. Microservices provide independently deployable services but add network failures, observability, deployment and consistency complexity.

Cross: When should you avoid microservices? When distributed complexity isn't justified.

92. API Gateway.

Model answer: It provides a common client entry point for routing, authentication integration, rate limiting, transformation and sometimes aggregation, while hiding internal topology.

Cross: Does it remove the need for service-to-service security? No.

93. Service discovery.

Model answer: Services locate healthy instances dynamically rather than hardcoding addresses. Examples include Eureka, Consul and Kubernetes service discovery.

Cross: Why needed? Scaling, restarts and dynamic placement.

94. Circuit breaker.

Model answer: It prevents repeatedly calling an unhealthy dependency. Typical states are CLOSED, OPEN and HALF_OPEN. Timeouts and bounded retries are often used with it.

Cross: Why use timeouts too? To avoid waiting indefinitely.

95. Microservice communication.

Model answer: Synchronous communication includes REST/gRPC. Asynchronous communication includes Kafka/RabbitMQ. Async decouples producers/consumers but introduces eventual consistency, retry, duplication and ordering concerns.

Cross: How do you handle duplicate events? Idempotent consumers/deduplication.

96. Process vs thread.

Model answer: A process has its own execution environment and memory space. Threads share process memory and resources, making them cheaper but requiring synchronization around shared state.

Cross: Define race condition: result depends on timing/interleaving.

97. Runnable vs Callable.

Model answer: Runnable does not return a result. Callable returns a result through Future and can throw checked exceptions.

Cross: How do you obtain the result? Future.get(), which may block.

98. synchronized vs volatile.

Model answer: synchronized provides mutual exclusion plus visibility guarantees. volatile provides visibility/order for a variable but does not make compound operations such as count++ atomic.

Cross: Safe counter? AtomicInteger or synchronized design.

99. ExecutorService.

Model answer: It manages task execution through reusable worker threads. It supports submit/execute and lifecycle methods such as shutdown. Pool sizing should reflect workload.

Cross: Why call shutdown? To release executor resources and allow clean lifecycle termination.

100. Simultaneous DB updates.

Model answer: Two users can overwrite each other's changes if concurrency is not controlled. Optimistic locking with @Version detects a version conflict; pessimistic locking uses database locks when stronger control is needed. Choose based on contention and business requirements.

Cross: Explain @Version: update includes the expected version; a stale version causes a concurrency conflict.

CODING + SQL ROUND — 15 PROBLEMS

1. Reverse a string

```
String input = "Infosys";
StringBuilder result = new StringBuilder();
for (int i=input.length()-1;i>=0;i--) result.append(input.charAt(i));
System.out.println(result);
```

O(n).

2. First non-repeated character

```
Character result = input.chars().mapToObj(c -> (char)c)
.collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()))
.entrySet().stream().filter(e -> e.getValue()==1).map(Map.Entry::getKey).findFirst().orElse(null);
```

LinkedHashMap preserves insertion order.

3. Find duplicates

```
Set<Integer> seen = new HashSet<>();
Set<Integer> duplicates = numbers.stream().filter(n -> !seen.add(n)).collect(Collectors.toSet());
```

Set.add returns false for an existing value.

4. Second-highest salary

```
Optional<Integer> secondHighest = employees.stream().map(Employee::getSalary).distinct()
.sorted(Comparator.reverseOrder()).skip(1).findFirst();
```

distinct handles duplicate salaries.

5. Highest salary by department

```
Map<String, Optional<Employee>> result = employees.stream().collect(
Collectors.groupingBy(Employee::getDepartment,
Collectors.maxBy(Comparator.comparing(Employee::getSalary))));
```

Know groupingBy + maxBy.

6. Employees above average salary

```
double avg = employees.stream().mapToInt(Employee::getSalary).average().orElse(0);
List<Employee> result = employees.stream().filter(e -> e.getSalary() > avg).toList();
```

Explain the two-pass approach.

7. Count by department

```
Map<String, Long> result = employees.stream().collect(
Collectors.groupingBy(Employee::getDepartment, Collectors.counting()));
```

Classic groupingBy + counting.

8. Sort salary descending

```
List<Employee> result = employees.stream().sorted(
Comparator.comparing(Employee::getSalary).reversed()).toList();
```

Use thenComparing for ties.

9. Partition even/odd

```
Map<Boolean, List<Integer>> result = numbers.stream()
.collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

true = even; false = odd.

10. Character frequency

```
Map<Character, Long> frequency = input.chars().mapToObj(c -> (char)c)
.collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Use LinkedHashMap supplier to preserve order.

11. SQL second-highest salary

```
SELECT MAX(salary) FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

For ranking with duplicates, know DENSE_RANK().

12. SQL count by department

```
SELECT department_id, COUNT(*)
FROM employee GROUP BY department_id;
```

Know GROUP BY.

13. SQL departments with >5 employees

```
SELECT department_id, COUNT(*)
FROM employee GROUP BY department_id
HAVING COUNT(*) > 5;
```

WHERE filters rows; HAVING filters groups.

14. SQL above department average

```
SELECT e.* FROM employee e
WHERE e.salary > (SELECT AVG(e2.salary) FROM employee e2
WHERE e2.department_id = e.department_id);
```

Know correlated subquery and window-function alternative.

15. Spring Boot GET by ID

```
@RestController
@RequestMapping("/employees")
class EmployeeController {
    private final EmployeeService service;
    EmployeeController(EmployeeService service){this.service=service;}
    @GetMapping("/{id}")
    ResponseEntity<EmployeeResponse> get(@PathVariable Long id){
        return ResponseEntity.ok(service.getEmployee(id));
    }
}

@Service
class EmployeeService {
    private final EmployeeRepository repository;
    EmployeeService(EmployeeRepository repository){this.repository=repository;}
    EmployeeResponse getEmployee(Long id){
        Employee e=repository.findById(id).orElseThrow(() ->
        new ResourceNotFoundException("Employee not found"));
        return mapToResponse(e);
    }
}

interface EmployeeRepository extends JpaRepository<Employee,Long>{}
```

Explain request → controller → service → repository → DB → DTO → JSON.

10 HIGH-PRESSURE SCENARIOS

API works locally but returns 500 in production

Read the first meaningful exception, compare profiles/configuration, database/external dependencies, deployment versions and recent changes. Fix root cause and add regression coverage.

API takes 8 seconds

Trace latency end-to-end. Check slow SQL, N+1, repeated queries, downstream calls, indexes, connection pools, thread pools and payload size. Measure before optimizing.

HashMap becomes slow

Investigate hash distribution, collisions, poor hashCode, mutable keys, resizing and incorrect concurrent access.

Duplicate POST

Use idempotency keys, unique constraints and transaction-safe deduplication. Retries are normal in distributed systems.

Database is down

Fail safely, log/alert, use bounded retries only where appropriate, circuit breaking/timeouts and a useful client-facing error.

Microservice repeatedly fails

Use timeouts, bounded exponential backoff, circuit breaker, health monitoring, tracing and fallback only where semantically valid.

Process one million records

Use DB pagination/streaming, batching, bounded concurrency and backpressure. Don't blindly use `parallelStream`.

Two APIs update same entity

Use optimistic locking with `@Version` or pessimistic locking depending on contention/business needs.

Stream code is slow

Check algorithmic complexity, repeated DB calls, boxing, unnecessary operations, huge data and inappropriate parallelism.

Create CRUD API from memory

Be able to build Entity → Repository → Service → DTO → Controller → validation → exception handler and explain every layer.

FINAL 25 — MUST KNOW COLD

1. Project explanation and one API end-to-end
2. OOP pillars
3. Overloading vs overriding
4. Java pass-by-value
5. String immutability
6. `equals`/`hashCode`
7. HashMap internals
8. HashMap vs ConcurrentHashMap
9. ArrayList vs LinkedList
10. Comparable vs Comparator

11. Java 8 functional interfaces
12. Streams
13. map/filter/flatMap
14. Optional + orElseGet
15. IoC/DI
16. Constructor injection
17. @SpringBootApplication
18. Auto-configuration
19. Controller-Service-Repository flow
20. DispatcherServlet
21. @Transactional + rollback
22. Lazy/eager + N+1
23. JWT + SecurityFilterChain
24. 401 vs 403
25. SQL second-highest salary

2-DAY INTERVIEW REHEARSAL

DAY 1 — Core + Spring: Questions 1–65. Answer aloud, then code questions 1–10. Build one CRUD API from memory without AI.

DAY 2 — REST/JPA/Security/Microservices: Questions 66–100. Practice SQL 11–14, the GET-by-ID API, and all 10 scenarios. Finish by explaining your actual project end-to-end in 3 minutes.

Interview rule: do not memorize definitions alone. For almost every answer, be ready for “Why?”, “How?”, “Give an example”, “What happens internally?” and “What did YOU do in your project?”